The Corridor Game

A GO / WAIT / DETOUR potential game for resolving conflicts between mobile sinks in the Dubins-TSP project

Theory, algorithm and evaluation of `corridor_game_gt.py` · 30 October 2026

In short

When two or more mobile sinks (MS) would come closer than the safe distance d_s, the sinks involved play a small game. Each chooses one of three strategies: **GO** (keep the plan), **WAIT** (stop briefly, then continue) or **DETOUR** (side-step and rejoin the own path; offered only when it is feasible).

Each sink pays a cost made of **collision risk** (seconds spent too close to another sink), **extra travel time**, **extra distance** and **waiting time**. Sinks switch to their best response one at a time until nobody can lower their own cost: a pure Nash equilibrium. The game is an **exact potential game** (Theorem 1), so this process always stops.

An equilibrium is **not** guaranteed to be collision-free. Section 8 separates what is proved from what is not; Section 9 reports experiments, which are evidence, not guarantees. Section 11 explains everything in plain words.

Contents. 1 Setting and notation · 2 When does a game start? · 3 The game: players, strategies, costs · 4 Payoff matrix for two sinks · 5 Best response and Nash equilibrium · 6 Potential function · 7 Algorithm · 8 Assumptions, guarantees, limitations · 9 Experimental results · 10 Changes compared with corridor_game.py · 11 The idea in plain words

1. Setting and notation

The planner (`main.py`, `visualize.py`) gives every mobile sink k a timed trajectory: one position $p_k[n] \in \mathbb{R}^2$ per frame $n = 0, 1, \ldots, F_k - 1$, with frames $\Delta t = 0.5$ s apart. Every tour starts and ends at the base station b. After its last frame a sink is treated as parked at its final position. `corridor_game_gt.py` changes trajectories only by inserting stops (WAIT) or by replacing a short piece with a side-step (DETOUR). Sensor assignment, visiting order and the planned routes are not changed.

Symbol	Meaning	In the code
K; $N \subseteq K$	all sinks with a trajectory; the players of one game	keys of `all_trajectories`; `players`
$p_k[n]$, F_k	position of sink k at frame n; number of frames	`trajs[k]`
Δt	time between frames (0.5 s)	`dt`
v, R	speed (5 m/s); minimum turning radius (5 m)	`env.v`, `env.R`
d_s	minimum allowed distance between two sinks (3 m)	`safe_dist`
b, r_b	base-station position; radius of its exempt zone (5 m)	`env.base_station`, `base_radius`
f, f_e	first and last interval of the conflict that starts a game	`f`, `f_end`
L; s, e	lead (5 frames); decision frame $s = \max(1, f - L)$; rejoin frame $e = f_e + L$	`lead`; `s`, `e`
A_k, a_k	strategy set of sink k ($\subseteq$ {GO, WAIT, DETOUR}); one strategy	`actions[k]`
a, a_{-k}	profile (one strategy per player); the strategies of everyone except k	`prof`
$p_k^{a_k}$	trajectory of sink k when it plays a_k	`Action.traj`
$u_{kj}[n]$	1 if sinks k and j are unsafe in interval n (Eq. 1)	`unsafe_intervals`

Symbol	Meaning	In the code
E_{kj}; B_k; ρ_k	risk exposure of a pair (Eq. 2); against non-players; risk of player k (Eq. 3)	`exposure`; `Game.background`; `Game.risk`
ΔT_k, ΔD_k, W_k	extra mission time, extra distance, waiting time	`extra_time`, `extra_dist`, `wait_time`
c_k; α; β, γ, δ	own cost (Eq. 4); collision weight; weights of time, distance, waiting	`Game.own`; `Game.alpha`; `weights`
J_k; $U_k = -J_k$	cost of player k (Eq. 5); its utility (payoff)	`Game.cost`
Φ	potential function (Eq. 8)	`Game.potential`
w_k, w_{max}	length of a WAIT in frames; its maximum (40 frames = 20 s)	`best_w`, `w_max`
h	side-step distance of a DETOUR, $h = d_s$ + 0.5 m = 3.5 m	`offset`
ε	numerical tolerance, 10^{-9}	`_EPS`

2. When does a game start?

Conflict test. Between frames n and $n+1$ both sinks are assumed to move in a straight line. With the relative position $r_{kj}[n] = p_k[n] - p_j[n]$, the closest distance during the interval and the unsafe indicator are

$$d_{kj}[n] = \min_{0 \le \tau \le 1} \left\| r_{kj}[n] + \tau \left(r_{kj}[n+1] - r_{kj}[n] \right) \right\| \tag{1a}$$

$$u_{kj}[n] = \mathbf{1}\left[\, d_{kj}[n] < d_s \;\text{ and }\; z_{kj}[n] = 0 \, \right] \tag{1}$$

Here $\mathbf{1}[\cdot]$ is 1 if the statement in brackets is true and 0 otherwise, and $z_{kj}[n] = 1$ when both sinks are within r_b of the base station at their moment of closest approach (the docking-zone exemption already used by `corridor_game.py`). Checking the whole interval, not only the sampled positions, also catches two sinks that pass each other between two frames.

Trigger. The program scans forward from frame n_0 (initially 1) for the earliest interval f in which $u_{kj}[f] = 1$ for some pair. The **players** N are all sinks that belong to such a pair. The conflict lasts until f_e, the end of the longest unsafe run (among those pairs) that starts at f. Decisions are taken $L = 5$ frames (2.5 s) earlier, at the decision frame $s = \max(1, f - L)$, so that there is room to stop or to swerve. All other sinks keep their current trajectories during the game. Afterwards the scan continues at $n_0 = f + 1$, so anything left over starts a new game.

3. The game: players, strategies and costs

One game is the tuple $G = (N, (A_k)_{k \in N}, (J_k)_{k \in N})$: the players, their strategy sets and their cost functions. It is a finite game with at most 3 strategies per player.

3.1 Strategies

Every strategy of sink k is a complete trajectory $p_k^{a_k}$ for that sink alone. All strategies, including the WAIT length and the DETOUR path, are built **before** the game starts and do not change during it. Theorem 1 relies on this.

- **GO**: keep the current trajectory, $p_k^{GO} = p_k$. Always available.
- **WAIT**: stop at the position of frame s for w_k frames, then continue on the unchanged path:

$$p_k^{WAIT}[n] = p_k[n] \;\; (n \le s), \qquad p_k[s] \;\; (s < n \le s + w_k), \qquad p_k[n - w_k] \;\; (n > s + w_k) \tag{2a}$$

The length $w_k \in \{1, \ldots, w_{max}\}$ is the shortest stop with the lowest number of unsafe intervals between frames s and $e + w_{max}$, counted against all other sinks as if they keep their current

trajectories. Ideally that number is zero: the sink waits just long enough to let the others pass. WAIT is available whenever the sink has not finished its tour. Stopping is assumed to be physically possible, as for the original "Yield".

- **DETOUR**: swerve h = 3.5 m to the side of the own path while the conflict lasts, then rejoin it. Let σ be the arc length along the path measured from frame s; σ_f, σ_e and σ_{end} its values at frames f, f_e and e; and $n(\sigma)$ the unit normal to the right of the direction of travel. The new path is

$$q(\sigma) = p(\sigma) + \chi\, h\, \varphi(\sigma)\, n(\sigma), \qquad \chi = +1 \text{ (right) or } -1 \text{ (left)} \tag{2b}$$

$$\varphi(\sigma) = 0.5(1 - \cos(\pi\sigma/\sigma_f)) \qquad \text{for } 0 \le \sigma < \sigma_f \tag{2c}$$

$$\varphi(\sigma) = 1 \qquad \text{for } \sigma_f \le \sigma \le \sigma_e$$

$$\varphi(\sigma) = 0.5(1 - \cos(\pi(\sigma_{end} - \sigma)/(\sigma_{end} - \sigma_e))) \qquad \text{for } \sigma_e < \sigma \le \sigma_{end}$$

The sink keeps right, like road traffic (χ = +1); the left side is used only if the right side is infeasible. Two sinks that meet head-on and both keep right therefore move apart ($2h$ = 7 m). The new path is re-timed at constant speed v, with points $v\Delta t$ apart. Where the original window contained short pauses at path joints (repeated frames), the same number of frames is kept, so a DETOUR is never faster than GO. DETOUR is offered only if all of the following hold:

Check	Condition	Why
F1 moving	the sink does not stand still for 2+ frames between s and f_e; a stop that starts after f_e just ends the detour earlier	a sink collecting data at a sensor cannot leave it
F2 ramps	$\sigma_f \ge l_{min}$ and $\sigma_{end} - \sigma_e \ge l_{min}$, with $l_{min} = \pi\sqrt{(hR/2)}$ = 9.3 m	room for a swerve that respects the turning radius
F3 turning radius	the discrete curvature (circle through three consecutive points) is ≤ 1.15/R everywhere on the new path	Dubins constraint; 15 % allowance for sampling (the planner's own paths reach 1.145/R)
F4 obstacles	the new path does not enter any obstacle inflated by d_{safe} = `env.d_safe`	stay outside the safety boundary
F5 budget (optional)	if `enforce_budget=True`: $(F_k^a - 1)\Delta t \le B$; also applied to WAIT	mission-time budget B = `env.budget`

Where l_{min} comes from. On a straight path the ramp is $y(x) = (h/2)(1 - \cos(\pi x/l))$. Its curvature satisfies $|y''|/(1 + y'^2)^{3/2} \le |y''| \le h\pi^2/(2l^2)$, which is at most $1/R$ exactly when $l \ge \pi\sqrt{(hR/2)}$. On curved paths this bound is not exact, which is why F3 checks the actual new path. GO is always kept, so A_k is never empty. Budget filtering (F5) is off by default, because many planned trajectories already exceed B (Section 9).

3.2 Cost functions

Collision risk. The exposure of a pair is the number of seconds the two sinks spend closer than d_s, from the decision frame on, computed on the trajectories that their strategies produce:

$$E_{kj}(a_k, a_j) = \Delta t \sum_{n \ge s} u_{kj}[n] \qquad \text{computed on } p_k^{a_k},\ p_j^{a_j} \tag{2}$$

The sum runs over all remaining intervals until both sinks have finished. E is symmetric, $E_{kj}(x, y) = E_{jk}(y, x)$, because Eq. (1) depends only on the distance between the two sinks and on both being in the base zone. It is always a whole multiple of Δt. A player's risk adds its exposure to the other players and to the fixed non-players:

$$\rho_k(a) = \sum_{j \in N,\ j \ne k} E_{kj}(a_k, a_j) + B_k(a_k), \qquad B_k(a_k) = \sum_{m \in K \setminus N} E_{km}(a_k, \text{GO}) \tag{3}$$

Own cost (all terms in seconds; distance is converted to the time needed to drive it):

$$c_k(a_k) = \beta\,\Delta T_k(a_k) + \gamma\,\Delta D_k(a_k)/v + \delta\,W_k(a_k) \tag{4}$$

Strategy	ΔT_k: extra mission time	ΔD_k: extra distance	W_k: waiting time
GO	0	0	0
WAIT	$w_k \Delta t$	0	$w_k \Delta t$
DETOUR	$(F_k^{\text{DETOUR}} - F_k)\Delta t \geq 0$	length of q − length of p between frames s and e	0

Total cost and utility of player k:

$$J_k(a) = \alpha\,\rho_k(a) + c_k(a_k), \qquad U_k(a) = -J_k(a) \tag{5}$$

$$\alpha = \left(1 + \sum_{k \in N}\left[\max_{x \in A_k} c_k(x) - \min_{x \in A_k} c_k(x)\right]\right)/\Delta t \tag{5b}$$

The terms overlap on purpose: a second of waiting is also a second of delay, and a detour's extra metres also take extra time. With the defaults β = γ = δ = 1, a second of waiting costs 2 and an extra metre costs $2/v = 0.4$. The weights express how much each nuisance matters. Theorem 1 holds for any values of β, γ and δ. The collision weight α is computed once per game from (5b), unless the user fixes it with `collision_weight`.

Lemma 1 (safety first, for each player)

With α from (5b), for every player k, every a_{-k} and all $x, y \in A_k$: if $\rho_k(x, a_{-k}) < \rho_k(y, a_{-k})$ then $J_k(x, a_{-k}) < J_k(y, a_{-k})$.

Proof. Risks are whole multiples of Δt, so $\rho_k(y,\cdot) - \rho_k(x,\cdot) \geq \Delta t$. Writing $\text{span}_i = \max_x c_i(x) - \min_x c_i(x)$: $J_k(y,\cdot) - J_k(x,\cdot) \geq \alpha\Delta t - \text{span}_k = 1 + \Sigma_i\,\text{span}_i - \text{span}_k \geq 1 > 0$. ∎

In words: a player never accepts more risk to save time, distance or waiting. If α is set by hand, the lemma holds only when $\alpha\Delta t > \text{span}_k$; the report records this as `safety_first`.

4. Payoff matrix for two mobile sinks

For two players 1 and 2 with no other sink nearby ($B = 0$), write $E(x, y) = E_{12}(x, y)$, $c_k^{\text{W}} = (\beta + \delta)w_k\Delta t$ and $c_k^{\text{D}} = \beta\Delta T_k^{\text{D}} + \gamma\Delta D_k^{\text{D}}/v$. Each cell lists the costs (J_1, J_2); the utilities are their negatives.

MS 1 \ MS 2	GO	WAIT	DETOUR
GO	$(\alpha E_{GG}, \alpha E_{GG})$	$(\alpha E_{GW}, \alpha E_{GW} + c_2^{\text{W}})$	$(\alpha E_{GD}, \alpha E_{GD} + c_2^{\text{D}})$
WAIT	$(\alpha E_{WG} + c_1^{\text{W}}, \alpha E_{WG})$	$(\alpha E_{WW} + c_1^{\text{W}}, \alpha E_{WW} + c_2^{\text{W}})$	$(\alpha E_{WD} + c_1^{\text{W}}, \alpha E_{WD} + c_2^{\text{D}})$
DETOUR	$(\alpha E_{DG} + c_1^{\text{D}}, \alpha E_{DG})$	$(\alpha E_{DW} + c_1^{\text{D}}, \alpha E_{DW} + c_2^{\text{W}})$	$(\alpha E_{DD} + c_1^{\text{D}}, \alpha E_{DD} + c_2^{\text{D}})$

Every cell is the shared danger $\alpha E(x, y)$, which **both** sinks pay in full, plus each sink's own nuisance. This structure is what makes the game a potential game: the potential of a cell is $\Phi(x, y) = \alpha E(x, y) + c_1(x) + c_2(y)$. With other sinks nearby each cost also contains $\alpha B_k(x)$; the function `payoff_matrix(game)` in the code includes it.

4.1 Example A: head-on encounter (numbers produced by the code)

Running `python corridor_game_gt.py`: two sinks drive towards each other on the same straight line at 5 m/s and would meet at $t = 10$ s (no obstacles, base station far away). The code builds WAIT = stop 1.0 s ($c^{\text{W}} = 2.00$) and DETOUR = side-step 3.5 m to the right with $\Delta T = 0.5$ s and $\Delta D = 1.13$ m ($c^{\text{D}} = 0.73$) for each sink, and $\alpha = (1 + 2 + 2)/0.5 = 10$. ★ marks pure Nash equilibria.

Costs (J_0, J_1)	MS 1: GO	MS 1: WAIT	MS 1: DETOUR
MS 0: GO	(10.00, 10.00)	(10.00, 12.00)	**(0.00, 0.73) ★**
MS 0: WAIT	(12.00, 10.00)	(12.00, 12.00)	(2.00, 0.73)
MS 0: DETOUR	**(0.73, 0.00) ★ ◀** reached	(0.73, 2.00)	(0.73, 0.73)

Potential Φ (rows MS 0, columns MS 1): GO: 10.00, 12.00, 0.73 · WAIT: 12.00, 14.00, 2.73 · DETOUR: 0.73, 2.73, 1.45.

The two equilibria (DETOUR, GO) and (GO, DETOUR) both have the lowest potential, 0.73. WAIT never helps here: a stopped sink is still in the other's way, and the cost shows it (the risk stays at 1.0 s). (DETOUR, DETOUR) is safe but not an equilibrium: if MS 1 already swerves, MS 0 saves 0.73 by driving straight. Starting from (GO, GO), both sinks would gain 9.27 by swerving; the tie goes to the sink with the shorter tour, then the lower id, so MS 0 detours (Figure 1a).

4.2 Example B: crossing paths

MS 0 drives east and MS 1 north; both would reach (50, 50) at t = 10 s. The strategies and α are the same as in Example A.

Costs (J_0, J_1)	MS 1: GO	MS 1: WAIT	MS 1: DETOUR
MS 0: GO	(10.00, 10.00)	**(0.00, 2.00) ★**	(10.00, 10.73)
MS 0: WAIT	**(2.00, 0.00) ★ ◀** reached	(12.00, 12.00)	(2.00, 0.73)
MS 0: DETOUR	(5.73, 5.00)	(15.73, 17.00)	**(0.73, 0.73) ★**

Potential Φ: GO: 10.00, 2.00, 10.73 · WAIT: 2.00, 14.00, 2.73 · DETOUR: 5.73, 17.73, 1.45.

There are three pure equilibria: (WAIT, GO) and (GO, WAIT) with Φ = 2.00, and (DETOUR, DETOUR) with Φ = 1.45. Best response from (GO, GO) reaches (WAIT, GO): MS 0 gains 8.00 by waiting, after which MS 1 has nothing to gain. This outcome is safe but not the cheapest equilibrium: best-response dynamics guarantees *an* equilibrium, not the best one. (WAIT, WAIT) shows the classic "both yield" failure: both stop, the meeting is only postponed, and both pay extra. The cost function penalises this without any special rule.

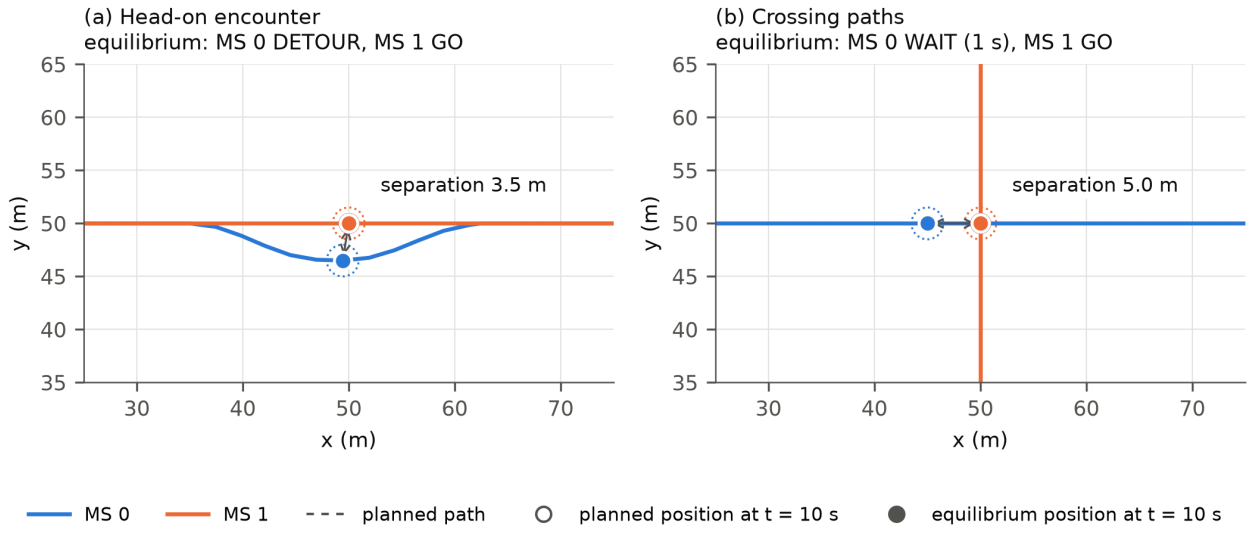


Figure 1. The equilibria reached in Examples A and B. Dashed: planned paths; open circles: planned positions at the moment of the planned collision (t = 10 s); filled circles: positions under the equilibrium. Produced by corridor_game_gt.py.

5. Best response and Nash equilibrium

The **best response** of player k to the strategies of the others is its cheapest strategy:

$$\mathrm{BR}_k(a_{-k}) = \operatorname{argmin}_{x \in A_k} J_k(x, a_{-k}) \tag{6}$$

In the code (`best_response`) ties keep the current strategy; otherwise the first of GO, WAIT, DETOUR is taken. A profile a^* is a **pure Nash equilibrium** if no player can lower its own cost by changing only its own strategy:

$$J_k(a^*) \le J_k(x, a^*_{-k}) \quad \forall k \in N, \ \forall x \in A_k \quad \Longleftrightarrow \quad a^*_k \in \mathrm{BR}_k(a^*_{-k}) \ \forall k \tag{7}$$

The code checks (7) explicitly after every game (`is_nash`) with tolerance $\varepsilon = 10^{-9}$ s.

Best-response dynamics (`best_response_dynamics`). Start from a^0 = (GO, …, GO). In each step every player computes its best response and its gain $g_k = J_k(a) - J_k(\mathrm{BR}_k(a_{-k}), a_{-k})$. If no gain exceeds ε, stop. Otherwise only the player with the largest gain switches. Ties go to the sink with fewer frames (the shorter tour, i.e. more spare time), then the lower id. The largest-gain rule tends to let the sink that can fix the conflict most cheaply act first. The shorter-tour tie-break keeps the idea of the original code that the less urgent sink gives way.

6. Potential function

$$\Phi(a) = \alpha\left[\sum_{k<j,\ k,j\in N} E_{kj}(a_k, a_j) + \sum_{k\in N} B_k(a_k)\right] + \sum_{k\in N} c_k(a_k) \tag{8}$$

Φ counts the danger of every pair of players once, the danger with non-players once, and every player's own nuisance.

> **Theorem 1 (exact potential game)**
> For every player $k \in N$, every profile a and every strategy $x \in A_k$:

$$J_k(x, a_{-k}) - J_k(a_k, a_{-k}) \ = \ \Phi(x, a_{-k}) - \Phi(a_k, a_{-k}) \tag{9}$$

Proof. Change only the strategy of k, from a_k to x. In Φ, (i) every pair term E_{ij} with $i, j \neq k$ is unchanged; (ii) every B_i and c_i with $i \neq k$ is unchanged, because they depend only on a_i (non-players are fixed). What remains are the pair terms that contain k, and B_k and c_k. A pair term containing k appears once in Φ, as E_{kj} (if $k < j$) or as E_{jk} (if $j < k$), and by symmetry $E_{jk}(a_j, a_k) = E_{kj}(a_k, a_j)$. Hence

$$\Phi(x, a_{-k}) - \Phi(a) = \alpha \sum_{j\in N,\ j\neq k} \left[E_{kj}(x, a_j) - E_{kj}(a_k, a_j)\right] + \alpha[B_k(x) - B_k(a_k)] + c_k(x) - c_k(a_k)$$

which by (3)–(5) is exactly $J_k(x, a_{-k}) - J_k(a)$. ∎

What the proof needs, and how the code ensures it.

	Condition	Ensured by
C1	Symmetric risk: $E_{kj}(x, y) = E_{jk}(y, x)$; a near-miss is charged equally to both sinks	Eq. (1) uses only the distance between the two and a symmetric base-zone test
C2	A strategy changes only its own sink's trajectory and is fixed before the game (WAIT length, DETOUR path)	`_build_actions` runs once, before `best_response_dynamics`
C3	Non-players keep their trajectories during the game	they enter only through B_k
C4	ΔT, ΔD, W depend on the own strategy only; waiting is always a sink's own choice	the table in Section 3.2
C5	α, β, γ, δ are constant during a game	α is computed once per game

If C1 failed, for example by charging a near-miss only to the follower, Eq. (9) would fail and the results below would no longer follow.

Corollary 1 (a pure equilibrium exists)

The game is finite, so Φ has a minimiser $a°$. If some player could lower its cost by deviating from $a°$, then by (9) Φ would fall below its minimum, which is impossible. So $a°$ is a pure Nash equilibrium. ∎

Corollary 2 (best-response dynamics always stops at an equilibrium)

Each move lowers the mover's cost by $g > \varepsilon$, so by (9) Φ falls by the same g. Φ strictly decreases along the run, so no profile is visited twice, and the run ends after at most $\Pi_k|A_k| - 1 \le 3^{|N|} - 1$ moves. It ends only when no gain exceeds ε, i.e. at a pure Nash equilibrium (7), up to ε. This holds whichever improving player is chosen to move; the largest-gain rule only affects *which* equilibrium is reached. ∎

The code raises an error if a run ever reaches $\Pi|A_k|$ moves, which would mean C1–C5 were violated. This has never happened (Section 9.4).

Lemma 2 (safety first, for the group)

Let $\Psi(a) = \Sigma_{k<j} E_{kj} + \Sigma_k B_k$ be the total risk inside Φ. With α from (5b): $\Psi(a) < \Psi(a')$ implies $\Phi(a) < \Phi(a')$.

Proof. $\Psi(a') - \Psi(a) \ge \Delta t$, and the sums of own costs differ by at most $\Sigma_k \text{span}_k$, so $\Phi(a') - \Phi(a) \ge \alpha\Delta t - \Sigma_k \text{span}_k = 1 > 0$. ∎

Consequence: every minimiser of Φ has the smallest total risk that any profile of this game can reach. In particular, if some profile removes all risk, so does every minimiser of Φ, and by Corollary 1 it is an equilibrium. The code does **not** search for this minimiser: best response stops at the first equilibrium it reaches, which can be worse (Example B, Section 8.3).

7. Algorithm

7.1 Workflow

1. Copy the input trajectories (the input is not modified) and record the conflict summary *before*.
2. Find the earliest unsafe interval $f \ge n_0$ (Eq. 1). Stop if there is none, or if `max_games` games have been played.
3. Players N = sinks that are unsafe at f; compute f_e, s and e.
4. Build A_k = {GO, WAIT} ∪ {DETOUR if F1–F4 (and F5)} for every player.
5. Compute the cost tables: E_{kj} for every pair of players and pair of strategies (at most 9 values), and B_k and c_k for every strategy. Set α by (5b).
6. Run best-response dynamics from all-GO until no player can gain (Eqs. 6, 7).
7. Replace each player's trajectory by its equilibrium strategy. Record the game: strategies, payoff matrix if $|N| = 2$, equilibrium, potential trace and the result of the Nash check.
8. Set $n_0 = f + 1$ and go to step 2.
9. Record the conflict summary *after*. Return the trajectories, plus the report if requested.

7.2 Pseudocode

```
procedure GAME_THEORY_RESOLVE(env, trajectories)
    n0 <- 1;  games <- 0
    while games < max_games:
        f <- earliest interval n >= n0 with u_kj[n] = 1 for some pair     # Eq. (1)
        if there is no such f: break
        N  <- { k : u_kj[f] = 1 for some j }                              # players
        fe <- end of the longest unsafe run starting at f (pairs unsafe at f)
        s  <- max(1, f - L);   e <- fe + L
        for k in N:  A_k <- {GO, WAIT(w_k)}  +  {DETOUR if F1..F4 (F5)}
        tables E_kj(x, y), B_k(x), c_k(x);   alpha <- Eq. (5b)
        a <- BEST_RESPONSE_DYNAMICS(N, A, J)
        for k in N:  p_k <- p_k^(a_k)
        n0 <- f + 1;  games <- games + 1
    return trajectories, report

procedure BEST_RESPONSE_DYNAMICS(N, A, J)
    a <- (GO, ..., GO)
    loop:
        for k in N:
            x_k <- argmin over x in A_k of J_k(x, a_-k)                   # Eq. (6)
            g_k <- J_k(a) - J_k(x_k, a_-k)
        if max_k g_k <= eps:  return a                                    # Eq. (7) holds
        k* <- argmax_k g_k        (ties: shorter tour, then lower id)
        a_k* <- x_k*                                                      # Phi falls by g_k*
```

7.3 Parameters (defaults of game_theory_resolve)

Parameter	Default	Meaning
`safe_dist`	3.0 m	d_s, minimum distance between two sinks (same default as the original)
`dt`	0.5 s	Δt, time between frames (as in `visualize.py`)
`weights`	(1, 1, 1)	(β, γ, δ) for extra time, extra distance, waiting time
`collision_weight`	None	α; None = automatic value (5b)
`max_wait`	20 s	longest WAIT ($w_{\max}$ = 40 frames)
`detour_margin`	0.5 m	side-step $h = d_s$ + margin = 3.5 m
`base_radius`	5.0 m	r_b, exempt zone around the base station (same as the original)
`enforce_budget`	False	F5: offer WAIT/DETOUR only within `env.budget`
`max_games`	500	upper limit on the number of games (termination)
`return_report`, `verbose`	False, True	also return the report; print a summary (2 = one line per game)
fixed	—	curvature allowance 15 %; lead $L = \lceil l_{\min}/(v\Delta t) \rceil + 1 = 5$ frames; $\varepsilon = 10^{-9}$

Complexity. With F the trajectory length: the scan before each game costs $O(|K|^2 F)$; choosing the WAIT length $O(w_{\max}|K|F)$ per player; the cost tables $O((9|N|^2 + 3|N||K|)F)$; each best-response step $O(3|N|^2)$ table look-ups. **Termination of the whole procedure:** each game stops (Corollary 2) and the number of games is capped by `max_games`. n_0 grows after every game, but WAIT and DETOUR can add frames, so n_0 alone would not bound the number of games; hence the cap. The report states whether it was reached (`hit_max_games`).

8. Assumptions, guarantees and limitations

8.1 Assumptions

	Assumption
A1	Time is discrete: frame n is time $n\Delta t$. In `visualize.py` consecutive frames are not exactly $v\Delta t$ apart (path joints repeat a frame, dwell times are rounded down), so this time is approximate.
A2	Between two frames a sink moves in a straight line (used by Eq. 1).
A3	A sink can stop and restart at once (stop-and-go), as the original "Yield" already assumed. A vehicle that cannot stop would need a loiter circle instead of WAIT.
A4	Trajectories are executed exactly as planned (open-loop timing).
A5	Two sinks that are both within r_b of the base station at their closest moment are exempt (docking zone), as in the original.
A6	A sink that has finished stays at its last position (the base station).
A7	Conditions C1–C5 of Section 6.
A8	One central program computes all decisions with full information. The "players" are a modelling device for coordinated planning, not independent on-board agents.

8.2 Theoretical guarantees (proved; hold for every input)

	Guarantee	Source
G1	Exact potential: $\Delta J_k = \Delta\Phi$ for every change by one player	Theorem 1
G2	Every game has a pure Nash equilibrium	Corollary 1
G3	Best-response dynamics stops after $\leq \Pi\lvert A_k\rvert - 1$ moves at a pure Nash equilibrium (tolerance ε)	Corollary 2; checked by `is_nash`
G4	With automatic α no player trades risk for time, distance or waiting; minimisers of Φ have the least achievable total risk	Lemmas 1, 2
G5	The whole procedure stops	`max_games`
G6	Every DETOUR satisfies F1–F4 (and F5 if enabled); every WAIT lasts at most `max_wait`	construction
G7	The input trajectories are not modified	construction

8.3 What is not guaranteed

- **A collision-free result.** An equilibrium can still contain risk when (i) no strategy removes it, for example a sink parked at a sensor right next to another sink's path, or (ii) the risk disappears only if two sinks change at the same time. Example of (ii): two players with {GO, WAIT}, $E = \Delta t$ in every cell except E(WAIT, WAIT) = 0, and c(WAIT) = 1. Then α from (5b) gives $\alpha\Delta t = 3$, so (GO, GO) costs 3 for each player and a lone switch to WAIT costs 4. (GO, GO) is therefore an equilibrium with risk, while (WAIT, WAIT) has no risk and a lower potential ($2 < 3$). Best response started at (GO, GO) stays there.
- **The best equilibrium.** Best response returns an equilibrium, not the minimiser of Φ (Example B: 2.00 instead of 1.45).
- **Consistency across games.** Games are solved one after another in time. A later game may change frames (from its own s) that an earlier game already examined. The cost discourages new risk anywhere after s, and the final summary measures the result, but frames before a game's own conflict are not re-examined.
- **The mission budget.** It is not enforced by default. Even with `enforce_budget=True`, only WAIT and DETOUR are filtered; GO keeps the planned trajectory, which may already exceed B.

- **Optimality of the whole plan**, and a polynomial worst-case number of best-response moves (only $3^{|N|} - 1$ is guaranteed).
- **Exact physical time** (A1).

8.4 Limitations (design choices)

- Each player gets one WAIT length and one DETOUR per game; longer waits or the other side are only reachable through later games.
- DETOUR is a sideways shift of the own path, not a new route through a different gap. It is checked (F1–F4), not optimised.
- Risk counts seconds below d_s, not how deep: 2.9 m and 0.1 m count the same.
- The weights β, γ, δ are modelling choices (default 1). α is set automatically.
- In `visualize.py` the static coloured lines show the planned paths; only the moving markers follow the trajectories returned by the game.

9. Experimental results (evidence, not guarantees)

> **Read this section differently**
> Everything in Sections 3–8 is proved for every input. The numbers below were measured on 20 random scenarios. They show how the method behaved there, not what it will always do.

9.1 Setup

Twenty scenarios were generated with the random generator of `main.py` (seeds 1–20: 10–15 obstacles, 90–110 sensors, $B = 80$, $v = 5$ m/s, $R = 5$ m, $d_{safe} = 2$ m). Sensors were partitioned and routed with `main.overall_algorithm`, and trajectories were built with `visualize.generate_interpolated_trajectory` ($\Delta t = 0.5$ s), giving 12–19 sinks per scenario. The repository's `ga_heuristic.py` currently fails to import (IndentationError at line 8 since commit a510f03), so the experiments used the last version of that file that compiles (commit 8b8e9b2). No repository file was changed. Three variants were compared: no resolution (the planned trajectories), `corridor_game.py`, and `corridor_game_gt.py` with default parameters. All three were scored by the same independent check, `conflict_summary`: Eq. (1) with $d_s = 3$ m and the 5 m base zone. The original code tests only sampled frames, so this check is stricter than its own test.

9.2 Results per scenario

Seed	K	Conflicting pairs plan / orig / GT	Risk exposure (s) plan / orig / GT	Closest GT (m)	Games	Wait (s) orig / GT	Extra dist. (m)	Makespan (s) plan → GT	Run time GT (s)
1	12	27 / 30 / **4**	68.5 / 59.0 / **4.5**	0.15	17	47.0 / 75.5	0.0	133.0 → 137.0	1.42
2	18	80 / 81 / **7**	214.5 / 224.0 / **8.0**	0.31	23	97.0 / 209.5	2.7	123.5 → 133.0	4.46
3	15	52 / 50 / **4**	109.5 / 74.5 / **2.5**	1.30	12	50.5 / 112.5	9.9	142.0 → 150.0	1.57
4	12	37 / 37 / **4**	58.0 / 53.0 / **3.0**	0.59	9	22.5 / 52.0	0.0	125.5 → 134.5	0.85
5	15	52 / 54 / **4**	105.5 / 86.5 / **3.5**	0.00	18	58.5 / 162.5	3.1	122.5 → 142.0	1.72
6	13	39 / 43 / **8**	94.0 / 63.0 / **9.5**	0.00	17	47.0 / 27.0	0.0	143.0 → 145.0	2.90
7	15	54 / 54 / **3**	124.5 / 94.5 / **2.5**	0.21	11	60.0 / 105.0	1.1	128.5 → 136.5	1.33
8	14	42 / 40 / **2**	100.0 / 97.0 / **3.0**	0.03	15	79.5 / 132.0	1.5	143.5 → 147.5	1.90
9	13	35 / 38 / **3**	91.0 / 69.5 / **2.5**	0.49	13	43.0 / 89.0	14.6	156.0 → 157.5	1.13
10	14	45 / 41 / **3**	117.0 / 107.0 / **3.0**	0.92	16	71.5 / 100.5	1.3	132.5 → 138.0	1.50
11	16	65 / 59 / **4**	128.5 / 96.5 / **3.5**	0.03	19	52.0 / 125.5	5.7	125.5 → 131.5	2.69
12	17	71 / 61 / **3**	138.5 / 90.0 / **1.5**	1.31	10	75.0 / 103.5	1.2	105.0 → 108.0	1.28

Seed	K	Conflicting pairs plan / orig / GT	Risk exposure (s) plan / orig / GT	Closest GT (m)	Games	Wait (s) orig / GT	Extra dist. (m)	Makespan (s) plan → GT	Run time GT (s)
13	13	54 / 51 / **2**	97.0 / 111.0 / **1.0**	1.30	9	74.0 / 88.5	8.1	143.5 → 163.0	0.89
14	14	56 / 55 / **5**	128.0 / 108.5 / **5.0**	0.01	12	64.5 / 99.5	9.5	128.5 → 135.5	1.57
15	16	56 / 52 / **6**	180.0 / 117.5 / **4.5**	1.01	16	58.5 / 107.5	22.7	124.5 → 134.5	2.17
16	16	49 / 50 / **3**	105.0 / 110.0 / **2.5**	0.09	10	75.5 / 168.5	0.0	128.0 → 141.0	1.78
17	19	89 / 84 / **8**	178.0 / 146.0 / **5.5**	0.16	21	86.0 / 134.0	10.9	95.5 → 108.5	2.95
18	17	60 / 54 / **1**	121.0 / 87.5 / **1.0**	0.77	10	57.0 / 113.0	2.3	99.5 → 107.5	1.49
19	14	37 / 39 / **1**	94.0 / 81.0 / **1.5**	0.01	10	53.5 / 80.0	0.0	136.0 → 160.0	1.12
20	15	53 / 54 / **4**	116.5 / 137.5 / **5.0**	0.04	22	91.0 / 166.5	4.2	123.5 → 141.5	2.51

plan = planned trajectories, no resolution; orig = `corridor_game.py`; GT = `corridor_game_gt.py`. "Closest" = smallest distance between two sinks outside the base zone. "Wait" for orig = frames it inserted × Δt. Makespan = longest tour duration.

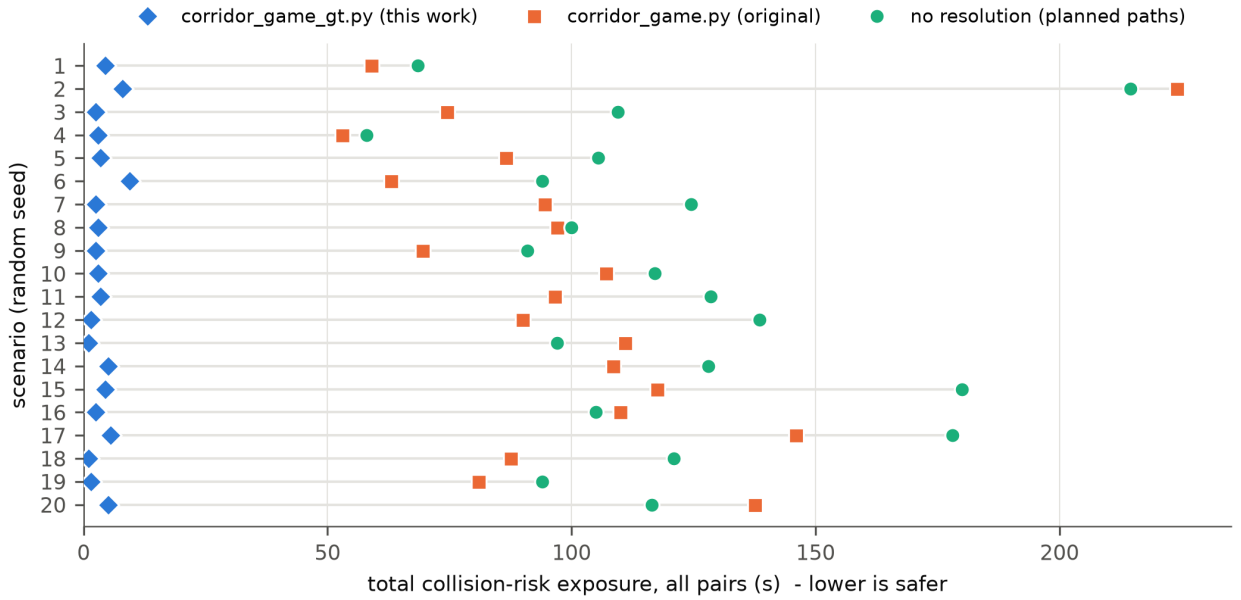


Figure 2. Total collision-risk exposure (seconds that pairs of sinks spend closer than 3 m) per scenario. Same data as the table in Section 9.2.

9.3 Summary

- **Risk exposure** fell from 58–214.5 s (median 113 s) for the planned trajectories to 1.0–9.5 s (median 3.0 s), i.e. by 90–99 % (median 97 %). `corridor_game.py` gave 53–224 s (median 95.5 s), between 18 % worse and 35 % better than doing nothing. Summed over the 20 scenarios: 2,369 s planned, 2,013.5 s original, 73 s new.
- **Conflicting pairs:** planned 27–89, original 30–84, new 1–8 (median 4).
- **Not collision-free.** No scenario ended without conflicts. The closest approach was 0.001–1.31 m (median 0.18 m), and below 1 m in 16 of 20 scenarios. Of the 146 remaining unsafe intervals, 49 % occur in the first 10 s and 55 % within 15 m of the base station, where all sinks leave together with the same initial heading; 29 % involve a sink standing still. Example: in scenario 1 at $t \approx 35$ s, MS 8 is collecting data at a sensor while MS 6 drives past at 0.8 m. DETOUR was not feasible for either sink, and no WAIT lowered MS 6's risk (by Lemma 1 it would otherwise have been chosen), so the equilibrium was (GO, GO) and the conflict was reported.
- **Price:** total waiting 27–209.5 s per scenario (median 106 s), against 22.5–97 s inserted by the original; extra distance 0–22.7 m (median 2.5 m); makespan +1.5 to +24 s (median +8 s),

against +0 to +1.5 s for the original.

- **Choices** in 893 player decisions: GO 496, WAIT 371, DETOUR 26. DETOUR was feasible in 105 decisions.
- **Games:** 9–23 per scenario (290 in total). The largest game had 12 players (all at departure). `max_games` was never reached.
- **Run time:** 0.85–4.46 s per scenario (median 1.57 s), against 0.05–0.12 s for the original.
- **Budget:** sinks whose tour exceeds $B = 80$ s: 7–11 per scenario before the game (median 9), 7–13 after (median 10.5). The pipeline already violates B: `main.py` checks the budget with straight-line distances, while the actual Dubins trajectories are longer. That is outside `corridor_game_gt.py`.

9.4 Numerical check of the theory

In all 290 games every profile and every single-player deviation was evaluated: 39,184 profiles and 829,614 deviations. The largest $|\Delta J_k - \Delta\Phi|$ was 2.3×10^{-13} s, which is floating-point rounding, as Theorem 1 predicts. Φ decreased strictly at every best-response move. Every final profile passed the Nash check (7). All pair tables were symmetric (C1), and the condition of Lemma 1 held in every game. Moves per game: none in 86 games, one in 140, two to nine in 64. These checks confirm the implementation on test data; they add no guarantee beyond Section 8.2.

10. Changes compared with corridor_game.py

Aspect	corridor_game.py (unchanged)	corridor_game_gt.py
Strategies	Go / Yield (a one-frame pause), decided per frame	GO / WAIT (a stop of w_k frames) / DETOUR (a side-step), decided per conflict
Payoffs	none; a priority score (list length + 10 × waits)	cost $J_k = \alpha\cdot\text{risk} + \beta\Delta T + \gamma\Delta D/v + \delta W$ (Eq. 5); 3×3 payoff matrix
Decision rule	the lower score yields	best-response dynamics to a verified pure Nash equilibrium
Interaction	pairwise decisions overwrite each other	all sinks in a conflict play one game; risk with the others counts too
Theory	none	exact potential game (Theorem 1), existence and convergence proved
Conflict test	positions at the same frame only	closest approach in each interval (also catches passes between frames)
Base zone	both within 5 m at that frame	both within 5 m at the closest moment
Deadlock rule	forced Go after 50 waits	WAIT ≤ `max_wait`; at most `max_games` games
Feasibility	not checked	DETOUR: turning radius, obstacles, not while parked; optional budget
Input	lists modified in place	copied; a new dict is returned
Output	trajectories	trajectories, plus an optional report (before/after, per-game strategies, payoff matrices, equilibria, Nash checks)

Interface. `game_theory_resolve(env, all_trajectories, safe_dist=3.0)` keeps its name, its arguments and its return type (a dict of lists of (x, y)). All new arguments are optional keywords. To use it in the animation, change one import in `visualize.py`:

```
from corridor_game import game_theory_resolve        # before
from corridor_game_gt import game_theory_resolve     # after
```

`python corridor_game_gt.py` prints the strategies and payoff matrices of Examples A and B.

11. The idea in plain words

The situation. Picture a few small robot carts in a big park. Each cart has its own round trip: it leaves the shed, stops at some letterboxes to collect mail (these are the sensors), and comes back. Sometimes two paths run close together and two carts could bump into each other. The program looks ahead, spots those moments, and decides what each cart involved should do.

The three choices. Think of two people walking towards each other in a narrow hallway. Each of them can:

- **GO**: just keep walking;
- **WAIT**: stop for a moment by a doorway and let the other one pass;
- **DETOUR**: step to the side (keep right, like cars do), pass, and step back.

Stepping aside is only allowed when there is room: not into a wall (an obstacle), not with a turn too sharp for the cart, and not while the cart is busy at a letterbox.

The bill. Every choice comes with a bill for that cart, with four lines: (1) **danger**: how many seconds the cart would be too close to another cart (closer than 3 metres), at a very high price per second; (2) **lateness**: how much later it gets home; (3) **extra walking**: extra metres, counted as the time needed to walk them; (4) **standing around**: seconds spent standing still. Every cart wants the smallest bill. That is all a "player" is: someone who wants a small bill.

What each formula says.

Formula	In plain words
(1) unsafe test	"Are these two too close right now?" A warning light for each pair of carts. It turns on when they come closer than 3 m at any moment, including between two snapshots, like watching the whole film rather than just the photos. Near the shed it is switched off, as in a car park.
(2) exposure	"For how many seconds is the light on?" Both carts in a pair get the same number: if I am too close to you, you are too close to me.
(3) risk	"Add up my danger with everybody."
(4) own cost	"My nuisance": lateness + extra walking + standing around, all in seconds.
(5) total cost	"My bill = danger × a big price + my nuisance." The big price α is set so that even half a second of danger costs more than any time that could be saved in this situation, like a fine for crossing on red that is bigger than any time you could gain. So a cart never chooses danger to save time (Lemma 1).
(6) best response	"Given what the others are doing, what is my cheapest choice?"
(7) Nash equilibrium	"Nobody wants to change their mind on their own." In the hallway: one person has stepped aside, the other walks on, and neither wants to swap.
(8) potential	"A shared scoreboard": the danger of every pair counted once, plus everybody's nuisance.
(9) Theorem 1	"Whenever one cart changes its choice, its own bill changes by exactly as much as the shared scoreboard." So every time somebody improves their own bill, the scoreboard goes down. A scoreboard that only goes down, and has only a limited number of possible values, cannot keep going down forever. So the back-and-forth always ends, and when it ends nobody wants to change: that is the equilibrium.

How the program uses this. It plays the whole plan forward like a film. At the first moment two or more carts would get too close, it pauses the film. The carts involved play the game, deciding a few seconds before that moment. Their choices are written into their plans, and the film continues to the next close call.

A worked example (Example A). Two carts meet head-on on one path. If both GO, each bill says 10: one second of danger at a price of 10. If cart 0 steps aside, its bill is 0.73 (half a second late and 1.1 m extra) and cart 1's bill is 0. Now nobody wants to change: cart 0 would be back at 10 by walking straight, and cart 1 cannot do better than 0. Had cart 0 only stopped, it would

still be in the way, and its bill would be 12.

What it cannot promise. Sometimes no choice helps. For example, a cart is busy at a letterbox right next to another cart's path: it cannot step aside, and making the other one stop does not help either. Sometimes the danger would only go away if two carts changed at the same moment, and neither will do it alone. In those cases the program reports the remaining danger instead of hiding it. In tests on 20 random maps it removed about 97 % of the danger time (median), but some close calls remained, mostly right after all carts leave the shed together. That is a measured result, not a promise.